

Cryptography: AES Benchmark Report

Assignment 2

Sagar Parajuli
CS26BTKMU11003

September 2026

Contents

1	Introduction	2
1.1	System and Compilation Environment	2
1.1.1	Hardware Configuration	2
1.1.2	Operating System	2
1.2	Compiler and Compiler Flags	3
1.2.1	Benchmark Considerations	3
2	Results	4
2.1	Benchmark Methodology	4
2.2	Benchmark Results	4
2.2.1	Throughput	4
2.2.2	Cycles per Byte	4
3	Discussion	5
3.1	Encryption Performance	5
3.2	Decryption Performance	5
3.3	Effect of Message Size	5
4	Conclusion	6
5	Appendix	7

1 Introduction

1.1 System and Compilation Environment

The AES cipher benchmarks were performed on the following hardware and software configuration. The linux commands 'neofetch', and 'lscpu', were used to fetch the system information.

1.1.1 Hardware Configuration

- **System:** Dell Inspiron 15 7000 Gaming
- **Processor:** Intel Core i7-7700HQ
- **Architecture:** $x86_64$
- **Physical Cores:** 4
- **Logical Threads:** 8
- **Threads per Core:** 2
- **Base CPU Frequency:** 2.80, GHz
- **Maximum CPU Frequency:** 3.80, GHz
- **L1 Cache:** 128, KiB data cache + 128, KiB instruction cache
- **L2 Cache:** 1, MiB
- **L3 Cache:** 6, MiB
- **Main Memory:** 15.86, GiB RAM

The processor supports several instruction-set extensions relevant to cryptographic implementations, including AES-NI, SSE2, AVX, and AVX2. In particular, AES-NI provides hardware instructions designed to accelerate AES encryption and decryption.

1.1.2 Operating System

- **Operating System:** Linux Mint 22.2 ($x86_64$)
- **Kernel:** 6.14.0-37-generic
- **Desktop Environment:** Cinnamon 6.4.8

1.2 Compiler and Compiler Flags

The AES benchmark was implemented in C and compiled using GCC version 13.3.0.

- **Compiler:** GCC 13.3.0
- **Compiler Distribution:** Ubuntu 13.3.0-6*ubuntu2* 24.04.1
- **Optimization Level:** -O3
- **AES Instruction Set:** -maes
- **SSE2 Instruction Set:** -msse2

The benchmark was compiled using the following command:

```
>>gcc -O3 -maes -msse2 -o benchmark benchmark.c
```

The -O3 flag enables a high level of compiler optimization. The -maes flag enables AES instructions, while -msse2 enables SSE2 instructions.

1.2.1 Benchmark Considerations

The processor supports AES-NI and the benchmark was compiled with -maes. Therefore, the AES-NI version can use hardware AES instructions during execution. The results for this version should therefore not be treated as measurements of a purely software AES implementation.

The same hardware and software configuration was used for all benchmark runs so that the implementations could be compared under the same conditions.

2 Results

The following throughput and cycle per byte outputs were obtained from the benchmark:

2.1 Benchmark Methodology

The benchmark results were obtained using the median of 31 timed runs, with 5 untimed warm-up runs discarded prior to measurement. CPU cycle counts were measured using serialized RDTSCP instructions, while wall-clock time was measured using CLOCK_MONOTONIC.

2.2 Benchmark Results

2.2.1 Throughput

Throughput is reported in megabytes per second (MB/s). Higher throughput indicates better performance.

Table 1: AES encryption and decryption throughput

Implementation	1 KB	2 KB	4 KB	32 KB	64 KB
Software Encryption	23.03	23.22	19.15	22.83	55.49
AES-NI Encryption	583.02	860.79	1048.38	1415.89	4949.32
Software Decryption	8.47	8.46	8.25	8.48	25.90
AES-NI Decryption	584.77	864.60	1006.77	1718.54	4971.76

2.2.2 Cycles per Byte

Performance in terms of CPU cycles per byte (CPB) is shown below. Lower CPB values indicate fewer CPU cycles required to process each byte.

Table 2: AES encryption and decryption cycles per byte

Implementation	1 KB	2 KB	4 KB	32 KB	64 KB
Software Encryption	113.32	113.86	138.92	117.14	48.22
AES-NI Encryption	1.71	1.66	1.78	1.78	0.52
Software Decryption	313.05	315.17	323.72	315.74	103.36
AES-NI Decryption	1.72	1.66	1.84	1.47	0.52

The benchmark output used `sink=0` to prevent the compiler from optimizing away the benchmarked computation.

3 Discussion

The results show that the AES-NI implementation is much faster than the software implementation. The difference can be seen in both throughput and cycles per byte (CPB), especially for the larger input sizes.

3.1 Encryption Performance

The AES NI implementation has much higher encryption throughput than the software version. At 1 KB, throughput increases from 23.03 MB/s to 583.02 MB/s, which is about a $25.3\times$ speedup. At 64 KB, the speedup increases to approximately $89.2\times$.

The CPB measurements show the same general trend. The software implementation requires between approximately 113 and 139 cycles per byte for small and medium input sizes, whereas AES-NI requires only around 1.7–1.8 cycles per byte. At 64 KB, the AES-NI implementation requires only 0.52 cycles per byte.

3.2 Decryption Performance

The difference is larger for decryption. At 1 KB, AES-NI gives 584.77 MB/s compared with 8.47 MB/s for the software implementation, giving a speedup of approximately $69.0\times$. The largest measured speedup is at 32 KB, where it is approximately $202.7\times$. At 64 KB, the speedup is approximately $192.0\times$.

This larger improvement is mainly due to the relatively poor performance of the software decryption implementation. The software decryption path requires significantly more cycles per byte than software encryption. For example, at 1 KB, software encryption requires 113.32 CPB, while software decryption requires 313.05 CPB.

AES-NI encryption and decryption have much more similar performance. At 64 KB, both require approximately 0.52 CPB, with throughputs of 4949.32 MB/s for encryption and 4971.76 MB/s for decryption. This suggests that the large encryption/decryption difference comes mainly from the software implementation, rather than from AES-NI itself.

The higher CPB for software decryption may be related to the inverse AES transformations, particularly *InvMixColumns*. This transformation uses the coefficients 9, 11, 13, and 14 in $\text{GF}(2^8)$, while the forward transformation includes coefficients such as 1. As a result, the decryption implementation used here performs more finite-field multiplication operations. This may contribute to the higher CPB measured for software decryption.

3.3 Effect of Message Size

The AES-NI implementation generally becomes more efficient as the message size increases. For encryption, throughput increases from 583.02 MB/s at 1 KB to 4949.32 MB/s at 64 KB. At the same time, CPB decreases from 1.71 to 0.52.

One reason for this may be the fixed overhead of each benchmark invocation. For small messages, function-call and timing overhead make up a larger part of the measured time. With larger messages, the same overhead is spread over more data, so its effect on the measurement becomes smaller.

The software implementation does not show the same pattern. Its CPB values stay fairly similar from 1 KB to 32 KB, but drop at 64 KB. For example, software encryption decreases from 117.14 CPB at 32 KB to 48.22 CPB at 64 KB.

The abrupt improvement at 64 KB should therefore be interpreted with caution. Possible factors include benchmark overhead, CPU frequency scaling, and differences in the amount of work performed per timing interval. Additional measurements at intermediate sizes, such as 8 KB and 16 KB, together with a larger number of trials, would help determine whether this behavior is a genuine performance characteristic or a measurement artifact.

4 Conclusion

The throughput and CPB results both show a large performance difference between AES-NI and the software implementation. The difference is larger for decryption because the software decryption implementation has a much higher CPB.

The two performance metrics also agree with each other: higher throughput corresponds to lower CPB. The speedups calculated from throughput and CPB are not exactly the same because throughput uses wall-clock time, while CPB uses processor cycle counts. CPU frequency scaling and other system effects can cause small differences between the two measurements.

On the tested Intel Core i7-7700HQ system, the software implementation required tens or hundreds of CPU cycles per byte, while the AES-NI implementation required approximately 0.5–1.8 cycles per byte. The measured throughput improvements reached roughly two orders of magnitude, showing the effect of hardware AES support on the benchmark.

5 Appendix

Code: <https://github.com/SagarP-0/Crypto2>

Report(tex): <https://www.overleaf.com/project/6ab3eae8d035b0a25486d63d/share#4da08cdb20a431fcb9a299e52b15424c85a2bb9946462f3>